Server Overview — A Space Data Network Node

Companions: Operator Enrollment (becoming an operator or admin — key generation, trust levels, exact HTTP calls); API Guide (the full REST surface, authentication, error semantics, anonymous data plane); `README.md` (getting started); TECH-PATH.md; `sdn-server/docs/gateway-api.md`. On any running node, the served API reference is at `http://<node>/api/v1/docs`.

> In the dashboard: Node · Identity · Peers · Store · Accounts — these links open the page beside this guide when it is read inside the dashboard's documentation panel.

1. What a node is

Run the binary and it describes itself:

```
$ spacedatanetwork --help
spacedatanetwork is a specialized fork of IPFS tailored for the Space Data Network.
It replaces generic content-addressed storage with FlatBuffer-native data handling
and SQLite-based structured storage, optimized for space data standards.
```

Two qualifications to that self-description:

- "Fork of IPFS": the server is a libp2p Go application (streams, DHT, pubsub, bitswap-style sync all over libp2p), and the repo carries an in-tree fork of Kubo, the IPFS reference implementation, hosting the node console and the IPFS side of publishing. Both derive their identity from the same node key.
- "SQLite-based structured storage" is legacy wording. The structured storage engine is **FlatSQL**, a SQL engine that runs as a WASM module and reads FlatBuffer records directly — no embedded SQLite in the stack. Details in Section 3.

A node is one long-running daemon per machine, reachable over libp2p (including through NAT via relay and hole punching), storing FlatBuffer records in append-only streams, serving them over HTTP and pubsub, and running WASM modules and flows for everything above socket plumbing.

Operational shape:

- One daemon per box. `daemon --help` exposes an `--allow-multi-daemon` flag marked "DEVELOPMENT ONLY"; production is a single instance managed as a background service (`init` , `start` , `status` , `stop` , `restart`).
- The CLI talks to the daemon over its local HTTP API (`status` prints `admin_url=http://127.0.0.1:7173/` by default; `open` prints the UI URL).
- Two topology tiers: full nodes on public IPs (DHT, relay, pin, publish) and light peers — including browsers — behind NAT. A browser running the SDN library is a peer, not a client: with the same mnemonic as a server node, it has the same cryptographic identity.
- Fleet software updates are pushed by signed update signals over an update feed; a node fetches, verifies, upgrades itself in place, and can roll back.

2. One format: SDS FlatBuffers end to end

The unit of data everywhere is a Space Data Standards FlatBuffer record, size-prefixed, in wire form. The same bytes that travel over pubsub are appended to disk, indexed in place, queried without deserialization, and served back out:

> network → disk → query → network is one format.

The node embeds 228 `.fbs` schema files (all the ratified Space Data Standards plus a few SDN-internal record schemas such as the publication log and ping records) and validates every record it stores against the matching schema. Two consequences:

- **Zero-copy reads.** Query results are aligned, size-prefixed FlatBuffer streams; a reader walks fields out of the wire bytes directly.
- **Keys match the schema exactly.** As JSON, field names match the IDL capitalization exactly (`NORAD_CAT_ID`, not `norad_cat_id`). Never re-case them at the edges.

Standards are neutral contracts: descriptions name capability classes, never vendors or organizations. Provenance — which archive, which measurement program — is carried as record data at runtime, not baked into the standard.

3. Storage: FlatSQL, append-only, engine-routed

Storage is **FlatSQL**: a SQL engine compiled to WASM, hosted in-process by the Go server (via the WasmEdge runtime), running the identical artifact the browser runs.

- **Durability on disk.** Tables and B-tree indexes are zero-copy indexes **over the record streams**: they store offsets into the FlatBuffer stream files, never copies of payload bytes. The engine writes files only through seven explicit, offset-addressed host imports, so an engine that cannot open real disk storage fails loudly rather than silently running in memory.
- **Append-only streams, journal as source of truth.** Records land in append-only stream files; nothing is rewritten in place. Boot is open + verify + tail over the record journal — the journal is the source of truth; the persisted index is a recovery accelerator that can cost time but never data.
- **Engine-routed by default.** Every embedded standard routes through the engine: each standard's records mirror into a per-source shadow table (`OMM@celestrak-gp`, for example) as stored. There are no per-standard hard-coded query paths; a query reads the standard's table. The canonical query shape for the latest records of a type is:

  ```
  SELECT _data FROM <TYPE> ORDER BY _rowid DESC LIMIT ?
  ```

  Two categories of standards are deliberately not routed: those whose IDL declares no file identifier, and those whose IDL declares an `(encrypted)` field. The latter are **sealed at rest** — the stream holds an encrypted envelope — so they are never mirrored into the public query surface, and their table names do not even resolve in SQL.
- **Per-record content addressing.** Records are content-addressed by SHA-256: the same FlatBuffer bytes always produce the same CID, so deduplication and network sync can trust byte equality.

The anonymous read surface is a schema property

What "anyone can read" is decided per **schema**, not per URL path, and it is fail-closed: a schema is anonymously readable only if deliberately on the allowlist. Today that is the public catalog and its provenance lane:

`OMM` (orbital element sets), `CAT` (catalog entries), `MPE` (mean parameter ephemerides), `SPW` (space weather indices), `RFB` (RF band specifications / emitter catalog), `LKS` (link status), `PNM` (publication notifications), `DPM` (dataset publication manifests), `EPM` (entity profile messages / identity records), `APP` (application package manifests), and `EGP` (entity groups).

Everything else — including `TBS` and `IQC` — is anonymous-read CLOSED by default until deliberately opened. Anonymous read never implies anonymous publish.

4. Identity and signatures

One HD key tree

A node's entire cryptography descends from one BIP-39 mnemonic, turned into seed via SLIP-10 with the standard BIP-44 path structure (coin type 0):

```
BIP-39 Mnemonic → seed → SLIP-10 Master Key
    ├── m/44'/0'/0'/0'/0'  →  Ed25519 Signing Key (also the libp2p PeerID)
    └── m/44'/0'/0'/1'/0'  →  X25519 Encryption Key
```

The signed identity is also the network identity: the Ed25519 signing key **is** the libp2p PeerID. Purpose-separated child keys follow the grammar `m/44'/0'/<account>'/<purpose>'/0'`, fully hardened at every level, so publishing a child's public half reveals nothing about the parent or its siblings. The purposes in use:

- **Identity signing** (purpose 0) — signs sign-in challenges, identity and publication records, dataset publications, module publications, and update manifests. Whatever this key signs, every node in the fleet will install and run.
- **Encryption** (purpose 1) — key agreement for encrypted records and module bundles; signs nothing.
- **Licensing grants** (purpose 2) — signs module-delivery and storefront access grants; clients verify grants against its public half.

At rest, the mnemonic is encrypted with a machine-derived key and fails closed if the machine changes. The managed IPFS repo identity is derived from the same root, never a separate key.

What is published vs kept private

The node publishes an identity record (EPM) with human and machine faces: a contact card with name, organization, and photo, plus the literal public keys that verify it — email aliases ride the vCard/QR forms as machine-read identifiers. The xpub and the derivation paths are private. Identity records are self-signed: the record's signatures must verify against keys carried in the record itself, and the card's primary signing key must be exactly the key the contact role names.

Domain-separated signatures

Any endpoint that can produce a signature over content the node did not author is a potential oracle for forging another protocol's signature — sign a raw SHA-256 of one thing and you may have signed something else. So every detached signature the node produces is domain-separated:

```
statement = domain-label || 0x00 || SHA-256(content)
```

Domains come from a closed registry (`SDN-MODULE-PUBLICATION-V1` , `SDN-UPDATE-MANIFEST-V1` , `SDN-UPDATE-SIGNAL-V1` , dataset publication bindings, and others). A signature minted for one statement kind can never be replayed as another; adding a new signed statement kind is a reviewed change, never a request parameter.

First login binds the key

A wallet first signs in with its Ed25519 key; that key is then bound to the identity (first use wins). A conflicting key presented later is refused. Possession of key material is thus proof of identity, in both directions: the node's own key is always admitted as an administrator.

5. Peers and operators are one kind of thing

From the CLI help, verbatim: "a node running somewhere and an account that logs in are the same kind of thing: a PRR-shaped identity carrying a trust level."

`accounts` is the command you use for both:

```
$ spacedatanetwork accounts --help
List and manage accounts (network peers and login operators are one thing)
```

- `accounts add` adds a **peer** by `--peer-id` , `--public-key` , or `--vcard` . It refuses an xpub on purpose: an xpub identifies a wallet account, not a libp2p host.
- `accounts list` shows the merged table: peer rows and operator rows, unified by the key that both derive (the secp256k1 account public key from an operator's xpub maps to a libp2p peer ID, which is what merges a peer row and an operator row for the same entity).
- `accounts trust` changes trust explicitly: `--peer-id` targets the peer; `--xpub` targets the operator. It only updates an existing row; it cannot create one.

Trust levels

Trust levels follow the PGP ownertrust scale, persisted as a signed integer:

LEVEL	VALUE	MEANING
`never`	-1	Deliberate distrust: a hard veto that no web-of-trust computation can override
`unknown` / `untrusted`	0	No assertion made; the fail-closed default
`marginal` / `limited`	1	Weakest positive assertion
`standard`	2	Between marginal and full; the default for ordinary peer access
`full` / `trusted`	3	Full confidence
`admin`	4	Operational super-user: operator sessions, admin API routes
`ultimate`	5	Reserved exclusively for the node's own identity; never granted to a remote peer or session

Assignable to operators up to `admin`; `ultimate` is the node's own key's level and `never` is not supported as an operator lockout. For peers, the full range applies. The node also computes a web-of-trust validity from the peer graph, which feeds effective trust for access decisions.

Web-of-trust detail: the peer graph records each peer's role (standard, bootstrap, relay, gateway, seed); connection admission is trust-aware; fleet connections (known bootstraps, configured trusted peers, pins) are protected from trimming.

How operator rows come to exist

Four ways, and it matters which one you use:

1. **Configuration**: a `users:` block in `config.yaml` (xpub, trust level, optional name and signing public key). Config entries win over database rows on every read, cannot have their trust changed through the API, and cannot be removed through the API.
2. **The dashboard**: the ACCOUNTS view's "Enroll a key" form (xpub, name, trust, signing public key in hex), which creates the row through the admin API.
3. **First-admin bootstrap**: if no admin exists anywhere, the next wallet that signs in is minted as the initial admin. The node therefore cannot be permanently locked out at the operator level by deleting the last admin row.
4. **The root ceremony**: the node's own key is always admitted as an admin, with or without a stored row.

Sign-in

No passwords. Sign-in is an Ed25519 challenge-response: `POST /api/auth/challenge` returns a random challenge; the wallet signs it; `POST /api/auth/verify` admits the session and sets an HttpOnly session cookie. The CLI authenticates the same way using the node's own root key, or accepts an existing session token via `--session-token` or `$SDN_SESSION_TOKEN`.

The full enrollment story — key generation, sign-in requirements, withdrawal rules — is in Operator Enrollment.

6. How data moves: libp2p, pubsub, IPFS

The network layer

Every node runs a libp2p host: TLS and noise for transport security, an autonomous system-style connection manager with trust-aware gating, resource management, NAT port mapping, hole punching, and client relay with automatic relay peer discovery. Full-public nodes participate in the DHT and can serve relay for light peers (relay is a config-gated, donated-bandwidth service). When serving TLS directly, the node advertises `libp2p.direct` addresses that browsers can dial.

Stream protocols:

- `/spacedatanetwork/sds-exchange/1.0.0` — record exchange
- `/space-data-network/id-exchange/1.0.0` — identity records
- `/space-data-network/chat/1.0.0`
- `/space-data-network/flatsql-sync/2.0.0/<op>` — record and published-shard sync (the high-throughput data lane); one protocol per op, `$QRP` frames only

Pubsub lanes

- **Per-schema data topics** — `/spacedatanetwork/sds/<SCHEMA>` — the streaming record lane: one topic per standard, joined for every validated schema. Publication notifications ride the PNM standard's own data topic, `/spacedatanetwork/sds/PNM.fbs`.
- **Feed heads** — `/space-data-network/feed-heads/2.0.0/<schema>` — small, signed, mutable pointer messages (one `$QRP` Page carrying one SEGMENT row). A feed head announces where a standard's latest immutable materialized set lives (record count, byte count, shard / query-index / manifest CIDs, publication CID) and is what replicas subscribe to *before* fetching the immutable shards.
- **Adjuncts** — `/spacedatanetwork/edge-relays` (relay announcements), storefront topics `/sdn/storefront/{listings,purchases,reviews}`, per-buyer delivery topics `/sdn/data/{listing_id}/{buyer_peer_id}`, and channel discovery topics.

The publication lifecycle (dataset lane)

Providers publish materialized datasets, and the whole lifecycle is signed end to end:

1. A provider ingests source data and normalizes it into SDS FlatBuffers.
2. Records are stored locally and mirrored into the engine's tables.
3. The provider exports the authoritative set as a `DATA_SHARD` (the streams) plus a `QUERY_INDEX` asset.
4. It builds and signs a dataset publication manifest (**DPM**) binding the file identifier, source and byte hashes, schema hash, and the assets' content addresses, and adds it to IPFS.
5. It signs and announces a publication notification (**PNM**) on the open pubsub topics.
6. Consumers resolve the latest PNM by peer + standard, verify the signature against the publisher's identity, fetch the manifest and assets by content address over libp2p, verify hashes, roots, and signatures, and import the shard into their own store.

Content addressing is what makes this safe without a trusted third party: bytes are their own addresses, and the signature chain binds names to bytes. Published-shard downloads run at near wire speed (the documented gate is 2 Gbit/s sustained, 99% of wire speed). Over HTTP, the same records are served against the per-schema anonymous allowlist of Section 3.

Pin policy

A node stores and serves its records and files by content address whether or not they are pinned. A pin is a separate decision: it keeps bytes out of IPFS garbage collection and in the set the node announces as pinned. By default a node pins only its own EPM and its account EPMs. Everything else is added to the node's blockstore unpinned until the operator turns its class on.

CLASS	COVERS	FEED
`NODE_EPM`	The node's own EPM. Always pinned.	—
`ACCOUNT_EPM`	The EPMs of the node's accounts. Always pinned.	—
`PUBLICATION`	Dataset publications: manifests, shards, indexes, PNMs, CAR bundles.	`<standard>/<provider>/<source>`, e.g. `OMM/celestrak/gp`
`SHARE`	Records published as open shares.	The standard, e.g. `OMM`
`FLOW_ASSET`	Files a flow or module adds or pins through its IPFS capability.	The flow or module id
`TIP`	Content announced in pubsub tips.	The tip topic
`STOREFRONT`	Storefront listings and their artifacts.	—
`ASSET_MODEL`	3D asset models.	—
`SITE_ASSET`	Homepage site assets.	—
`PROFILE_PHOTO`	Profile photos of the node and its accounts.	—

A rule turns a whole class on or off, or one feed within it; a feed rule overrides its class rule. The operator sets them in the SDN UI's pin menu, which reads and writes the node's policy as an SDS `$PIN`:

- `GET /api/node/pins` answers one size-prefixed `$PIN`: every rule, whether it is on, whether it is locked, and the roots and bytes it holds pinned.
- `POST /api/node/pins` takes one size-prefixed `$PIN` carrying only the rules to change (`CLASS`, `FEED`, `ENABLED`) and answers the whole policy. Turning a rule on pins what the node already holds under it, from its own blockstore. Turning it off unpins what the rule pinned, unless another enabled rule holds the same content.

Both routes are operator only and use `application/vnd.sdn.flatbuffers.stream` with `X-SDN-Schema: PIN.fbs`. A change to `NODE_EPM` or `ACCOUNT_EPM` is refused. A lane's retention rule (`ReplaceCurrent`, `KeepAll`, `ArchiveAll`) decides what the node's store keeps; it never pins. Unpinned bytes stay in the blockstore until the operator runs IPFS garbage collection, which the node never runs on its own.

7. Everything compute is WASM; the host only connects

All functionality is WASM. The Go host and the browser JavaScript host are thin, application-blind connector layers exposing the same small set of generic hooks (network, storage, file I/O, signing). A module compiled once runs identically under WasmEdge on a node and in a browser tab, threaded through the wasi-threads contract.

Divergence between the two runtimes is treated as a defect, caught by running identical scenarios against both artifacts.

In practice:

- **The SQL engine is itself a WASM module** hosted in-process by the Go server, using the same FlatSQL artifact the browser uses (byte-parity between the artifacts is enforced by shared test vectors). The server-side build is the no-exceptions variant, because the AOT compiler must be able to precompile it; interpreted execution is far too slow for query work. Long-running engine calls are bounded by an execution timeout.
- **HTTP routes are flows.** The public record-retrieval surface (`/api/v1/data/...`) is compiled WASM flow bundles mounted by configuration. The host does request plumbing and nothing else:

  > There is no gateway: the only host glue is socket plumbing with zero decisions — HTTP request → one request frame → response frames written verbatim; all routing, query parsing, format selection, caching, and ETag logic live inside the WASM flow. Which flow owns which listener path is configuration, never Go code.

  Requests are serialized per flow instance; each mount runs a small pool of instances (default 4).
- **Ingestion runners are flows too.** The credentialed source workers (Space-Track, UDL, and friends — `ingest` in the CLI) run as flows over the host's timer, HTTP egress, and guarded persistence, not as custom Go fetchers.
- **Host functions are few.** The shared set is essentially `clock_now_ms` , `random_bytes` , and `log` . Everything else a module can do comes through the capability bridge.
- **Capabilities are explicit grants.** A module's access to storage is granted (and refused) as named capabilities — `storage.write` (write a record of a schema) vs `storage.query` vs `storage.delete` . Write does not imply read; a module that cannot load cannot touch anything.

The browser SDK follows the same boundary: a browser peer hosts the same wasm-threaded modules, so a flow behaves identically whether its host is a server daemon or a page.

8. Apps and the store

Apps

An "app" is a loadable runtime module or a timer-served flow bundle — the ingest workers are apps in this sense. The app registry treats an app entry as data like anything else: an installed entry is *decoded from* a signed app-package FlatBuffer record (`APP`), never invented from metadata, and the node's own dashboard record is built from the artifact bytes it serves, so the record and the served page can never disagree. `apps list` reads the public app feed; `apps run` fires an app's schedule on demand.

A served app page is self-contained by contract: CSS and JavaScript inlined, assets as data URIs, zero external requests. Pages served by a node load nothing from external origins — the sibling API Guide assumes the same rule for anything you mount.

The marketplace / store

- Listing, purchase-request, and review records (`STF` , `PUR` , `REV`) live on dedicated pubsub topics and in the store; the catalog is DHT-broadcast; delivery of purchased data goes over per-buyer topics.

- Canonical discovery is the `PLG` manifest published on spacedatastandards.org: exactly one signed listing per plugin ID and version, so the network can agree on what "the plugin `X` version `Y`" means.
- Module delivery is a signed handshake: discover the manifest, connect to the provider peer, complete a challenge/grant exchange verified against the licensing-grant key advertised in the provider's identity, fetch the encrypted WASM bundle by content address, unwrap the content key, decrypt locally, and load it through the standard module harness. Nothing server side ever runs the module or sees the content key.
- Encrypted record streams advertise their own field-level policy in-band: `FSM` records mark each field Public / Encrypted / Redacted / Unavailable, so a consumer can handle a record it cannot decrypt correctly.

`marketplace` in the CLI filters listings by kind (`data_stream` or `wasm_module`), access type (`one-time` , `subscription` , `streaming` , `query`), standard, tags, and provider ID.

9. How the pieces fit

Follow one record from origin to a consumer's screen:

1. **Origin.** A provider (human, script, or flow worker) acquires source data — say, an orbital element set — and encodes it as a size-prefixed SDS FlatBuffer (`OMM`). The bytes are now the record; there is no other form.
2. **Validation and admission.** `ingest` -style workers or the publish API present the bytes. The node checks the schema is allowed for publishing on this node, checks the session/trust/authorization policy, caps the body size, and validates the record structurally against the schema. The record's own size prefix and file identifier decide which standard's table it enters — a caller can never steer an `OMM` record onto another standard's table through an allowed path.
3. **Storage.** The bytes are appended to the standard's append-only stream (content-addressed, CID-deduplicated), indexed, journaled, and mirrored into the engine's per-source shadow table for the standard. On disk: one stream, one engine, one format.
4. **Announcement.** A signed feed head goes out on the schema's feed-heads topic; the record may also stream on the schema's pubsub topic. The provider periodically materializes the authoritative set into a shard + query index, signs a DPM, pins it to IPFS, and signs a PNM.
5. **Propagation.** Other nodes verify the feed head / PNM signatures against the publisher's published identity key, fetch the shard by content address over `flatsql-sync` , verify hashes and signatures, and import it. Full nodes with public addresses serve and pin; light peers and browsers sync over relay.
6. **Serving.** Records answer queries from the engine: the public schemas over the anonymous HTTP data plane, everything else behind authentication. A requesting browser is itself a node — it fetched, verified, and stores the same bytes locally rather than trusting the server's word.

That is the whole design: one format, one identity tree, signed at every boundary, with the host acting only as a connector.

10. Where to go next

- **Operator Enrollment** — become an operator or admin: key generation, `derive-xpub` , the dashboard enrollment form, trust levels, withdrawal and lockout rules, and every exact HTTP call.
- **API Guide** — the REST surface in full: authentication, the anonymous allowlist, write paths, error and refusal semantics, and CORS/same-origin behavior.
- On a running node: `http://<node>/api/v1/docs` serves the generated OpenAPI reference for exactly the routes that node mounted (anonymous, same-origin).
- TECH-PATH.md — the authoritative statement of the architecture, with rationale and the order data flows.
- `sdn-server/docs/gateway-api.md` — the gateway API design and implementation record.

- `README.md` — installation, quick start (`init`, `start`, `status`), and components.

Not yet supported

- No CLI command creates an operator (xpub) row. Enrollment happens through the dashboard, `config.yaml`, the first-admin bootstrap, or the node-root ceremony — see Operator Enrollment for each path and its exact behavior.
- Anonymous publishing is not enabled by default on a node; it is an explicit opt-in configuration, and even then records are attributed to an untrusted principal.
- The bulk record-retrieval routes present JSON for the orbital-element standard only; other standards answer in the native streaming FlatBuffer format. JSON is a read-side presentation choice, not a second wire format.

Space Data Network — Interaction Guide

Commands below ran against a live local daemon at `http://localhost:7173` (suite 1.0.4, standards 1.186.0) unless noted.

Local nodes listen on `http://127.0.0.1:7173` by default; `spacedatanetwork status` prints the admin URL, `spacedatanetwork open` opens it.

Public immutable downloads can also use an enabled publisher's full peer-ID subdomain. See Peer-ID URLs and the public cache for the working CelesTrak example, CID verification, native discovery, and the distinction between cached artifacts and this node's paginated query API. The in-app API panel and `/api/v1/docs` include the same public example.

---

1. Verify a node is alive and read data anonymously

```
# 1. Who is this node?
curl http://localhost:7173/api/v1/version
# -> {"agent_version":"spacedatanetwork/1.0.5","kubo_version":"0.39.0",
#     "standards_version":"1.217.0","suite_version":"1.0.5"}
# kubo_version is the upstream Kubo this node RUNS (suite.versions.json
# kubo.shipped). It used to print "0.40.0-dev", read from the in-repo fork,
# which no release path has ever shipped.

curl http://localhost:7173/api/v1/id
# -> {"peer_id":"16Uiu2HAku...","listen_addresses":["/ip4/...","..."],
#     "agent_version":"...","suite_version":"...","standards_version":"..."}

# 2. What does this node hold, and what are its rate limits?
curl http://localhost:7173/api/v1/catalog
# -> {"peer_id":"...","capabilities":["data_query","data_publish","pubsub"],
#     "rate_limits":
{"max_record_bytes":10485760,"publish_per_minute":10,"query_per_minute":1000},
#     "schemas":[{"name":"TBS.fbs","record_count":1078755,"total_bytes":758000000},
...]}

# 3. The anonymous record index (CIDs + a few indexed columns, no content).
#    The answer is ONE $QRP{KIND=Page} FlatBuffer frame
#    (application/vnd.sdn.flatbuffers.stream: [u32le length][FlatBuffer]):
curl -o page.qrp "http://localhost:7173/api/v1/data/index?schema=TBS&limit=3"
# -> TOTAL_COUNT 1078755, PAGE 1, LIMIT 3, CURSOR, NEXT_CURSOR, STALE false,
#    AS_OF_MS; COLUMNS [cid, norad, epoch_ms]; each ROWS[i].CELLS holds the
#    cid (Text) and norad and epoch_ms (Int, or Null), in store order. Read on
#    with &cursor=<NEXT_CURSOR> until NEXT_CURSOR comes back empty.
#    sdn-js: decodeQrpIndexPage(bytes).

# 4. Health check and the human-readable API reference (both anonymous):
curl -o health.qrp http://localhost:7173/api/v1/data/health
# -> one $QRP{KIND=Page} frame: COLUMNS component, status, time; one row

curl http://localhost:7173/api/v1/docs
# -> 200 text/html  (the Gateway API reference UI; zero off-node requests)
```

If step 1 fails, the daemon is not running or not on that port. Record CONTENT is a separate gate from the index (section 2).

---

2. The anonymous surface

The auth wall is default-deny: `/api/**` admits an unauthenticated request only in three cases.

What is anonymous

1. **Unconditional POST** (no session, no CSRF):
   - `POST /api/auth/challenge`, `POST /api/auth/verify` — sign-in ceremony (section 3); public by design (no session yet).
   - `POST /api/storefront/listings/search`
   - `POST /api/storefront/payments/stripe/webhook`
2. **GET/HEAD of the public read paths** (verbatim source: `sdn-server/cmd/spacedatanetwork/main.go`, `isPublicReadAPIPath`):
   - `GET /api/v1/version`, `/api/v1/id`, `/api/v1/stats`
   - `GET /api/node/info` — full public identity: peer_id, versions, EPM-derived addresses, chain proofs, listen addresses, onion address.
   - `GET /api/node/epm`, `/api/node/epm/json`, `/api/node/epm/vcard`, `/api/node/epm/qr`
   - `GET /api/relay/status` — peer_id, connections, configured_nodes, max_connections, load, mode, versions, uptime_seconds.
   - `GET /api/v1/catalog`, `/api/v1/data/health`
   - `GET /api/v1/data/index?schema=<CODE>&limit=&cursor=` — anonymous CID index, answered as one `$QRP` page frame in store order (at most 20,000 rows a page; a larger `limit` is clamped). It pages as `/api/v1/data/query` does (*Querying records*, section 4).
   - `GET /api/v1/pubsub/topics`, `/api/v1/pubsub/messages?schema=&limit=`
   - `GET /api/v1/channels`, `/api/v1/channels/<standardCode>`
   - `GET /api/apps`, `/api/v1/apps/default`, `/api/v1/apps/records/<...>` — app feed and app record bytes.
   - `GET /api/storefront/listings`, `/api/storefront/listings/<anything>`, `/api/storefront/trust/`
   - `GET /api/v1/trust/bond` — the node's security bond.
   - `GET /api/module-delivery/provider`, `/api/module-delivery/listings` — the provider descriptor and module listings.
   - `GET /api/v1/docs`, `/api/v1/openapi.json`, `/api/v1/docs/<asset>` — the API reference itself is anonymous.
   - `GET /api/auth/status` — `{"admin_configured","root_admin_available","users_configured","config_path","wallet_ui_configured", ...}`; tells whether a node has an operator yet.
   - `GET /api/directory/...`, `/api/v1/demo/...`, `/api/v1/log/...`
   - `GET /api/v1/peers` — legacy native peer list.
   - `GET /sdn/libp2p.js`

3. **Per-schema record content** — `GET /api/v1/data/<CODE>/bulk` — by allow-list, below.

The fail-closed allow-list: public readability is a property of the standard

`/api/v1/data/<CODE>/bulk` serves record CONTENT, gated by a schema-classified allow-list, not the path; a schema is anonymous only if named in it. Fail-closed: a new standard is NOT readable until deliberately allow-listed — the right default for a store that also holds key material, access grants, node-internal ledgers.

Public: `OMM` (orbital element sets — the primary catalog), `CAT` (catalog entries), `MPE` (mean parameter ephemerides), `SPW` (space weather indices), `RFB` (published emitter catalog), `LKS` (link status), `PNM` (publication notifications — already broadcast on public pubsub topics), `DPM` (dataset publication manifests — already pinned publicly), `EPM` (entity profile messages — the identity records), `APP` (application package manifests), `EGP` (entity groups — membership assertions over already-public catalog records).

NOT public: `TBS` (telemetry bitstreams), `IQC` (integripulse quality check) — both answer `401` (verified live):

```
curl -i "http://localhost:7173/api/v1/data/tbs/bulk?limit=1"
# -> HTTP/1.1 401 Unauthorized
#    one $QRP{KIND=Error} frame: ERROR_CODE unauthorized, HTTP_STATUS 401

curl -i "http://localhost:7173/api/v1/data/iqc/bulk?limit=1"
# -> HTTP/1.1 401 Unauthorized
#    one $QRP{KIND=Error} frame: ERROR_CODE unauthorized, HTTP_STATUS 401
```

- Accept spelling is flexible: `OMM`, `omm`, `OMM.fbs` all resolve.
- The anonymous INDEX is served for ANY schema: listing CIDs is not reading records. Verified live: the TBS index above queried 1,078,755 records anonymously; the TBS content route still refused.
- The allow-list governs READ only. Publishing is a separate, authenticated surface (section 4).

Reading record content: bulk

Served by the node's data-retrieval engine (a WASM flow inside the node, not a plain Go endpoint):

```
curl "http://localhost:7173/api/v1/data/<CODE>/bulk?limit=100&epoch=2026-08-
27T00:00:00Z"
```

Parameters: `epoch` (RFC3339 or unix seconds; default now), `profile` or `mode` (`nearest` / `as_of` / `forward`; default per node config), `limit` (min 1; default unbounded), `source` (a registered provider source), `format` (`flatbuffer` default; `json` accepted by the OMM route only).

Response: size-prefixed FlatBuffer stream (8-byte aligned `[u32le length][bytes]` frames; content type `application/vnd.sdn.flatbuffers.stream`) or, for OMM `format=json`, a bare JSON array with schema-exact keys (`NORAD_CAT_ID`, `EPOCH`, `MEAN_MOTION` — never lowercased). Both encodings carry a record count header and an ETag; send `If-None-Match` for `304 Not Modified`.

Failure modes:

- `401` — standard not on the allow-list (TBS, IQC, every not-yet-listed standard). Permanent by design; not a retry condition.
- `503 "flow unavailable: engine hot window not ready"` — standard IS public but the query engine has not hydrated its hot window yet (observed verbatim on the authoring node for `omm/bulk` and `pnm/bulk`).

Warm-up: retry with backoff. Light routes (`/api/v1/catalog` , `/api/v1/data/index` , `/api/v1/data/health`) stay live meanwhile.

---

3. The authenticated surface — signing in as an operator

Ed25519 challenge-response with a wallet key — no passwords, no bearer tokens. Identity: BIP-32 xpub + Ed25519 signing public key (64 hex chars). The signing key signs; the xpub is the account address the operator row is stored under.

The ceremony

```
TS=$(date +%s)
# The challenge request carries the signing public key and, optionally, the xpub.
# ts must be within 2 minutes of the server clock (verified: outside that window
# the answer is 400 {"code":"invalid_timestamp", ...}).
curl -X POST http://localhost:7173/api/auth/challenge \
  -H 'Content-Type: application/json' \
  -d "{\"xpub\":\"xpub6...\",\"client_pubkey_hex\":\"<64-hex Ed25519 public
key>\",\"ts\":$TS}"
# ->
{"challenge_id":"cca6041ac1...","challenge":"q4XAAWmLAvjCTZDBeBuwSA4UKNLr3DKSmCV/WAJrlG0
",
#     "expires_at":1787856891}

# Then sign the raw challenge bytes with the Ed25519 private key and:
curl -X POST http://localhost:7173/api/auth/verify \
  -H 'Content-Type: application/json' \
  -d '{"challenge_id":"...","challenge":"...","client_pubkey_hex":"<64-
hex>","xpub":"xpub6...","signature_hex":"<128-hex Ed25519 signature of the raw challenge
bytes>"}'
# -> 200, Set-Cookie: sdn_wallet_session=...; HttpOnly; SameSite=Lax; [Secure]
#    {"user":
{"xpub_fingerprint":"...","name":"...","trust_level":"admin"},"expires_at":...}
```

Session facts (verified from source):

- Cookie `sdn_wallet_session` : `HttpOnly` , `SameSite=Lax` , `Secure` over TLS or `X-Forwarded-Proto: https` . Legacy name `sdn_session` also recognized.
- Challenges single-use, 60 s expiry; limits 60/min per IP and 30/min per xpub (challenge), 120/min and 60/min (verify).
- Unknown identities get a real-looking challenge that can never verify — the node deliberately does not store it, so no enrollment enumeration. Verified live: a made-up xpub returned `200` with a valid `challenge_id` ; verification would fail.
- Key binding is first-use-wins (TOFU): the first signing key at a successful verify binds permanently to the xpub; a conflicting key is refused later. An admin can pre-seed the signing key when creating the operator row.
- The node's own root account signs in at admin level without any store row. With NO operator at all (no config users, no database rows), the first wallet that completes a verify becomes the "Initial Admin" (first-admin bootstrap). A BIP-32 master xpub (depth 0) is refused here — it would enumerate the whole wallet.

How to call as an operator from a script

The CLI demonstrates the two sanctioned ways:

1. Auto sign-in as the node's root key: every operator command (`spacedatanetwork accounts list` , `... status` , `... providers` , ...) completes the full challenge-verify ceremony with the node's own key and reuses the session cookie.
2. Pass an existing session token explicitly:

```
export SDN_SESSION_TOKEN="<token>"
spacedatanetwork accounts list
# or on every command:
spacedatanetwork accounts list --session-token "$SDN_SESSION_TOKEN"
```

There is no way to mint a session token without completing the challenge-verify ceremony once.

What an operator can see and do

Trust levels: `never` , `unknown` , `marginal` , `standard` , `full` , `admin` , `ultimate` (seven values).

- Anonymous + sign-in reads: anything in section 2, plus `GET /api/auth/me` (who you are, at any trust tier).
- `standard` (default floor for publishing): write data (section 4).
- `admin` : everything, including the operator matrix (`/api/auth/users`), peer trust management (`/api/peers`), the admin data surface (`/api/v1/data/scan|query|records|...`), imports/exports, and `GET /api/accounts` (the merged account view).
- `ultimate` is reserved for the node's own identity. Operator rows: `unknown` through `admin` , never `never` (lockout) and never `ultimate` ; both refused with `400 {"code":"invalid_trust_level"}` .
- Below the required tier: `403 {"code":"forbidden"}` .

Operator life cycle:

- Operator rows are created by: the node's `users:` config block, the dashboard's "Enroll a key" form (`POST /api/auth/users` , admin-only), the first-admin bootstrap, or the root sign-in's record row.
- `spacedatanetwork accounts trust --xpub <xpub> --level <level>` UPDATEs an existing row (`PUT /api/auth/users/<xpub>`); never creates one. A nonexistent xpub answers `400 {"code":"update_failed","message":"user not found"}` .
- Config-managed operators cannot have trust changed or be removed through the API.

---

4. Writing data to a node (vendor ingest)

Publish surface: `POST /api/v1/data/publish/{schema}` and `POST /api/v1/data/publish/batch/{schema}` . Mounted only when the node's config enables publishing; requires a session. Anonymous attempts are refused at the wall (verified: `401 {"code":"unauthorized"}`).

Admission path, in the exact order the node checks it

1. Publishing enabled? No → `403 data publishing is disabled on this node` .
2. Schema from the URL path: missing or invalid → `400` .

3. Schema in the publishing allow-list (config; empty list = all)? No → `403 schema not allowed for publishing: <schema>`.
4. Session present? No → `401 no session`.
5. Trust gate (default `standard`, node-configurable). Below → `403`.
6. ABAC policy (if configured): denied → `403 access denied by policy: <reason>`.
7. Body over max record bytes (10 MiB default) → `413 request body too large`.
8. Record's OWN size prefix + FlatBuffer file identifier decide the schema; mismatch with the URL path → `400` naming both. A caller can never reach another schema's table through an allowed path.
9. Structural FlatBuffer validation → `400 validation failed: ...`.
10. Storage quota → `403`.
11. Stored → `201 {"cid":"<sha256 hex>","schema":"<canonical>","stored_at":"<RFC3339>","bytes":N}`.

Batch mode: same ladder, body of repeated `[u32le size][flatbuffer]` frames, total capped at 10 times the max record bytes, per-frame results in the response. Single record body: raw size-prefixed FlatBuffer bytes of the standard (content type `application/vnd.sdn.flatbuffers` or similar binary media — not JSON).

Optional provenance query parameters: `?source_name=...&provider_id=...&batch_id=...&source_url=...` — tagged records feed the per-source, per-batch progress aggregates on `/api/v1/stats`; untagged publishes keep the legacy path.

Streaming

The same `[u32le size][flatbuffer]` framing streams in both directions.

Reads: `GET /api/v1/data/<CODE>/bulk` streams frames as they are written (`X-SDN-Stream-Format: flatsql-size-prefixed-le-u32`); consume incrementally, one chunk in memory at a time. sdn-js: `transport.streamData({schema})` returns an async frame iterator; `iterateSizePrefixedFrameStream(source)` decodes any byte stream in the same wire format.

Writes: `transport.publishBatchStream(schema, records)` takes any sync or async iterable of record buffers and carries it in size-bounded batch requests (default 8 MiB each, under the server's batch body cap), aggregating the per-record results. Unbounded sources publish without buffering the whole set.

Scanning a lane

`POST /api/v1/data/scan` (admin, unless the operator lists it in `admin.public_paths`) pages through every stored record of one standard, optionally one lane. It is FlatBuffers only: the request body is one size-prefixed `$QRP{KIND=Request}` frame with content type `application/vnd.sdn.flatbuffers.stream`, and JSON is refused with `415`.

- Request fields: `SCHEMA_NAME` (required); `PROVIDER_ID`, `SOURCE_NAME`, `BATCH_ID`, `PRODUCER_PEER_ID`, `ORIGIN_ID` (the storing peer), `SEARCH`, `LIMIT` (at most 50,000) and `CURSOR` (optional).
- Response: runs of one `$QRP{KIND=Page}` header followed by exactly `RECORD_COUNT` record frames, the stored bytes verbatim. Each header names its lane (`PROVIDER_ID`, `SOURCE_NAME`, `BATCH_ID`, `PRODUCER_PEER_ID`, `ORIGIN_ID`) and repeats the page fields (`CURSOR`, `NEXT_CURSOR`, `TOTAL_COUNT`, `LIMIT`, `ETAG`, `SCHEMA_NAME`, `FILE_IDENTIFIER`). An empty page is one header with `RECORD_COUNT` 0.
- A record's CID comes from its bytes: CIDv1, raw codec, sha2-256 (`bafkrei...`), the id `/api/v1/data/records/<schema>/<cid>` serves.
- Paging: send `CURSOR` = `NEXT_CURSOR` until it comes back empty. The cursor is opaque; `ETAG` identifies a page.

- Resume: the last page of a complete scan (`NEXT_CURSOR` empty) carries a `RESUME_CURSOR` on every run header. Send it back as `RESUME_CURSOR`, never with `CURSOR`, with the same query (schema, lane fields, search): the answer holds only the rows stored after that scan, byte-identical and in store order, paged by `NEXT_CURSOR`, and its last page carries a fresh `RESUME_CURSOR`. With nothing new it is one empty page and a fresh cursor. `TOTAL_COUNT` is 0 (unknown) on resumed pages. The cost follows the new rows, not the feed, and the cursor survives restarts. It is opaque and only the node issues it.
- Errors: one `$QRP{KIND=Error}` frame with `ERROR_CODE`, `MESSAGE` and `RETRY_AFTER_MS`. `STATUS` `Busy` or `Cold` means retry after that delay. `410` `resume_cursor_expired` (the store was rebuilt or compacted) means drop the cursor and scan again from the first page. `400` `resume_cursor_query_mismatch`, `invalid_resume_cursor` and `bad_request` (`CURSOR` with `RESUME_CURSOR`) refuse a resume as sent.

sdn-js: `encodeQrpScanRequest`, `decodeQrpScanStream`, `decodeQrpIndexPage` and `decodeQrpError` (`sdn-js/src/ui/runtime/qrp-frames.ts`).

Querying records

`POST /api/v1/data/query` (admin) selects records with conditions. The body is one size-prefixed `$QRP{KIND=Request}` frame; JSON is refused with `415` and `GET` with `405` (`method_not_allowed`).

- Fields: `SCHEMA_NAME` ; the lane fields `PROVIDER_ID`, `SOURCE_NAME`, `BATCH_ID`, `PRODUCER_PEER_ID`, `ORIGIN_ID` ; `SQL` (the sync-filter grammar); `SEARCH` ; `FROM_EPOCH_MS` and `TO_EPOCH_MS` ; `TIME_PROFILE` (`All`, `Nearest`, `AsOf`, `Latest`) with `PROFILE_EPOCH_MS` ; `FILTERS` (`NORAD_CAT_ID`, `ENTITY_ID`, `MAX_DELTA_SECONDS`); `PAGE` (1-based); `LIMIT` (at most 20,000); `CURSOR` ; `ARCHIVE_ID` .
- The answer has the scan's shape: runs of a `$QRP{KIND=Page}` lane header and its `RECORD_COUNT` records, verbatim.
- Order: store sequence ascending, the scan's order, so a batch comes back in the order it was stored. There is no other order: `SORT_FIELD` or `SORT_DESC` is refused with a `$QRP` Error `400` .
- The first page (no `CURSOR`) pins a snapshot at the lane's sequence head. It holds the snapshot's records `[(PAGE-1)×LIMIT, PAGE×LIMIT)` , and `TOTAL_COUNT` counts the snapshot.
- Every page header carries `CURSOR` and `NEXT_CURSOR` , both opaque and issued by the node. `NEXT_CURSOR` is empty on the snapshot's last page. Send it back as `CURSOR` with the same query: the next `LIMIT` records come from the same snapshot, `PAGE` is not used, and `TOTAL_COUNT` stays the same. A cursor walk returns every record of the snapshot exactly once, even while ingest or expiry runs. Records stored after the first page are not in it.
- `PAGE` without a cursor pins a fresh snapshot for each page, so `TOTAL_COUNT` can grow. Sequences only grow, so a `PAGE` walk is exact under append but not under expiry. Walk with the cursor for an exact read.
- Refused with a `$QRP` Error `400` : a malformed cursor, a cursor issued for another lane or query, and `CURSOR` on a `TIME_PROFILE` query or on a small-body (`SBD`) query with an `SQL` field filter. Those two keep their own order and page by `PAGE` : a time profile in the epoch index's profile order, a small-body filter in sequence order.
- The index (`GET /api/v1/data/index`) pages the same way, with the cursor as `&cursor=` . Its rows (`cid` , `norad` , `epoch_ms`) come in the query's order.
- Time profiles ride on the query; there is no separate epoch route. A time profile needs the schema's epoch profile: a schema without one answers `400` .

The node's status routes answer one `$QRP{KIND=Page}` each, with `COLUMNS` and typed `ROWS` (cells `Text`, `Int`, `Float`, `Bool`, `Bytes` or `Null`): `GET /api/v1/data/health`, `/api/v1/data/summary`, `/api/v1/data/datastores`, `/api/v1/data/local-replica-stats` and `/api/v1/data/sources` . In `summary` , the lane rows come first, then one total row per schema (lane fields empty), then the store total (`schema_name` empty).

`POST /api/v1/query` (admin) runs SQL in the node's sandbox: a `$QRP` Request with `SQL` and `PARAMETERS`, answered by one `COLUMNS` + `ROWS` page, with blobs as `Bytes` cells.

sdn-js: `src/ui/runtime/qrp-request.ts` builds the requests and `src/ui/runtime/qrp-frames.ts` decodes the answers.

What is verified — and what is not

Verified on admission: session identity, trust tier, policy, quota, and structural FlatBuffer validity (well-formed, correct file identifier, schema conformance).

Individual records are not signed or encrypted, by design. Authentication is the ingest gate: once a session is admitted and authorized to publish, the records it sends need no per-message signature.

Signing happens at the publication layer instead. `$PNM` is the digital signature message: it carries `SIGNATURE` over the `CID`, `TIMESTAMP_SIGNATURE` over the publish time, the `SIGNATURE_TYPE` naming the scheme, and `MULTIFORMAT_ADDRESS` — the canonical location where the corresponding data is found. A PNM rides in a stream of messages or travels on its own. For dataset updates the CID points at a `$DPM`, which carries the full verification contract: provider identity, retrieval protocol, canonical query, result hash, Merkle roots, and `FILE_ID`.

One signature attests a whole publication, bound to its content address — not one signature per record.

No JSON/NDJSON bulk-write endpoint: batch is size-prefixed FlatBuffer frames only. JSON is a read-side presentation format only.

Rate limits: per-minute publish rate (this node advertises 10/min in `/api/v1/catalog`) and per-publisher storage quotas.

The node also runs credentialed first-party acquisition (`spacedatanetwork ingest` — Space-Track and UDL, each with its own credentialed sign-in) and scheduled flow-based ingestion; separate lanes from the vendor HTTP API above.

---

5. Refusal semantics — what a client should do

HTTP status first, body shape second. On the data routes (`/api/v1/data/*`, `/api/v1/query` and the rest of the data plane) a refusal is one `$QRP{KIND=Error}` frame (`ERROR_CODE`, `MESSAGE`, `STATUS`, `RETRY_AFTER_MS`, `HTTP_STATUS`), and the HTTP status matches it. Elsewhere the auth wall answers `{"code","message"}`.

STATUS	BODY (REAL EXAMPLES)	MEANING	WHAT THE CLIENT SHOULD DO
200	`$QRP` index page; catalog rows; bulk stream; `{"status":"ok"}`	Success	Nothing
201	`{"cid":"...","schema":"...","stored_at":"...","bytes":N}`	Record stored	Trust the CID; use it for reference
304	empty	`If-None-Match` hit on a conditional read	Use your cached copy
503	`$QRP` Error frame: `STATUS` Busy, `ERROR_CODE` `STORE_BUSY`, `RETRY_AFTER_MS`	The store is busy (index, scan)	Retry after `RETRY_AFTER_MS`
400	`{"code":"invalid_timestamp","message":"timestamp outside allowable skew"}`	Bad request: skew, malformed body, route/header schema mismatch, invalid trust level, unknown operator on config-managed row	Fix the request; do not retry unchanged
401	`$QRP` Error `unauthorized` on data routes; `{"code":"unauthorized","message":"not authenticated"}` elsewhere	No or expired session; OR standard not on the anonymous allow-list (bulk reads)	TBS/IQC bulk: give up, permanent by design. Otherwise: sign in (section 3)
400	`$QRP` Error `resume_cursor_query_mismatch`, `invalid_resume_cursor`, `bad_request`	A scan resume refused as sent	Resend the same query with the node's own `RESUME_CURSOR`; never build one
410	`$QRP` Error `resume_cursor_expired`	The store was rebuilt or compacted since the cursor was issued	Drop the cursor; scan again from the first page
415	`$QRP` Error `unsupported_media_type`	A JSON body on a FlatBuffer route	Send one size-prefixed `$QRP` Request

STATUS	BODY (REAL EXAMPLES)	MEANING	WHAT THE CLIENT SHOULD DO
403	`{"code":"forbidden"}` ; `data publishing is disabled on this node` ; `schema not allowed...` ; `access denied by policy...` ; `CSRF validation failed (...)`	Below required trust tier, or a policy/CSRF refusal	Elevate trust or change lanes. CSRF: by design for cross-origin browsers — use the documented request signing scheme or a server-side caller
404	`$QRP` Error `not_found` on data routes (a record miss); `{"error":{"message":...}}` elsewhere	No such route / record / flow param	Check the path; never retry blindly
413	`request body too large`	Over max record bytes (10 MiB default)	Shrink or split the payload
429	`{"code":"too_many_requests",...}`	Challenge/verify or publish rate limit	Back off; respect `Retry-After` if present
503	`flow unavailable: engine hot window not ready` (plain text)	Standard is public but the query engine has not hydrated its hot window	Retry with backoff — warm-up, not denial; light routes stay live
503	`local storage unavailable in edge mode`	Edge mode, no local store	Read-only node

State-changing browser requests with a session cookie are CSRF-gated: same-origin Origin, Referer, or `X-Requested-With` (the CLI sends `X-Requested-With: XMLHttpRequest`). Anonymous public routes skip CSRF — safe for cross-origin reads.

---

6. For AI agents — discovering capability at runtime

Do not hardcode this document's lists; they change as standards are added and flows are mounted. Discover instead:

1. **Who the node is**: `GET /api/v1/id` then `GET /api/node/info` — peer_id, agent/suite/standards versions, the identity record.
2. **What it holds and its limits**: `GET /api/v1/catalog` — `capabilities` (`data_query`, `data_publish`, `pubsub`), per-schema `record_count` / `total_bytes`, `rate_limits` (`query_per_minute`, `publish_per_minute`, `max_record_bytes`).

3. **The generated API reference**: `GET /api/v1/openapi.json` (strong ETag — send `If-None-Match`; 304 comes back cheap). Read the stamps:
    - `x-sdn-served-by: flow|native` and `x-sdn-anonymous: true|false` per operation, stamped with the EFFECTIVE decision at boot.
    - Unstamped entries are planned routes not mounted yet.
    - The spec is not a complete route map — the light read surfaces (`/api/v1/catalog`, `/api/v1/data/index`, `/api/v1/version`, `/api/v1/pubsub/topics`, ...) live outside it. Cross-check both sources.
    - The human-readable form of the same spec is `GET /api/v1/docs` (self-hosted UI; its configuration explicitly disables external fonts and network agents).
4. **Public readability is a property of the schema, not of a path.** Probe the content route, read the refusal:

    ```
    curl -i "http://localhost:7173/api/v1/data/<CODE>/bulk?limit=1"
    # 200      -> public, and the engine is hot
    # 503      -> public, engine still warming (retry with backoff)
    # 401      -> not on the anonymous allow-list; content stays behind the gate
    ```

    The only durable definition of public is the allow-list itself (`sdn-server/internal/sds/public_read.go`); it is fail-closed, so when in doubt, probe.
5. **Prefer reads with ETags**: bulk reads and the spec carry ETags or record counts; conditional requests with `If-None-Match` get cheap 304s. Keep the ETag from the first response and replay it.
6. **Writing**: same admission ladder as a human (section 4). If you are an agent given an operator session token, send it as the `sdn_wallet_session` cookie (or use the node's CLI, which proxies the ceremony for you). Do not sign individual records — authentication is the gate, and attestation is a `$PNM` over the publication's CID (section 4).

---

7. Not yet supported

- Any JSON/NDJSON bulk-write endpoint. Batch publish is size-prefixed FlatBuffer frames only.
- Any bearer-token scheme. `sdn_wallet_session` is HttpOnly by design; the deliberate escapes are the root-key CLI ceremony and `$SDN_SESSION_TOKEN`.
- `spacedatanetwork identity new` — referenced in some example configs but not present in the current binary; `spacedatanetwork derive-xpub` derives an xpub from an existing BIP-39 mnemonic; key generation happens in the browser wallet ceremony.
- A CLI command that creates an operator row: `accounts trust --xpub` updates only (creation paths: section 3).
- TLS "origin" mode: TLS is `disabled`, `static`, or managed (automatic certificates). HTTP stays plaintext in local/dev modes.
- `/api/v1/standards`, `/api/v1/peers/{peerId}/pnm`, `/api/v1/peers/{peerId}/{standard}/latest`: planned gateway routes served only when the corresponding flow bundle is mounted on a node. Unstamped in the OpenAPI spec today; probe before relying on them.
- The `peers-discovery` endpoints (described in the API reference) are served by optional flow mounts; may not exist on every node — probe first.

8. Reference — what requires a session

Everything under `/api/**` not listed in section 2 requires a session. `admin`-gated examples: `/api/v1/data/scan`, `/api/v1/data/query`, `/api/v1/query`, `/api/v1/data/records/<schema>/<cid>`, `/api/accounts`, `/api/peers/*` — including `/api/peers/sdn`, `/api/peers/graph` and `/api/peers/graph/schema`, all of which answer `401` without a session, `/api/auth/users`, `/api/v1/conjunction/screen`, `/api/import`, `/api/export`, `/api/v1/modules/runtime/`.

The same classification logic — a single predicate — drives the auth wall, CORS, and the OpenAPI generator's anonymity stamps.

OPERATOR ENROLLMENT

1. Operators and peers: one kind of thing with a trust level

A **peer** is a network identity: a libp2p peer ID that talks to this node over the network, carries data, and can be granted a trust level. An **operator** is a login identity: a BIP-32 account extended public key (an **xpub**) recorded in the node's user store, used with a wallet to sign in to the dashboard. Every operator derives a peer ID from its xpub, so one identity can be both a network participant and a login account.

`accounts list` shows the merged view — one row per identity, whether enrolled as a peer, an operator, or both:

```
$ spacedatanetwork accounts list
{
  "accounts": [
    {
      "can_sign_in": true,
      "kind": "operator",
      "name": "Node Root",
      "peer_id": "16Uiu2HAkuSSuf8u32gYjS24jmER35Le5GFYLJfJeaQNfPEWPsJuA",
      "source": "database",
      "trust_level": "admin",
      "xpub": "xpub6CfBgD57CQrTjuj9FDcVjvfwmu2beKsKqTYLt2rQFsm6..."
    },
    {
      "can_sign_in": false,
      "kind": "peer",
      "name": "...",
      "peer_id": "16Uiu2HAm1LbvwjEHW2GDP2ZQZvwHLZrz2jbYoRLQmJEQ3wZ5Fm45",
      "trust_level": "full"
    }
  ]
}
```

A merged row has two underlying records (a peer entry and an operator entry), so every trust change must say which one it targets — the CLI is explicit about this (Section 6).

Two identities are special:

- **The node's own key** (the root derived from the node's seed). Whoever holds the seed can always sign in as administrator, with or without any user-store row. It appears in `accounts list` as "Node Root".
- **The first admin bootstrap** (Section 5): a node with no administrators mints the first sign-in as "Initial Admin".

2. The key handover, end to end

An operator row needs **two public values** before it can ever sign in:

FIELD	WHAT IT IS	WHERE IT COMES FROM
`xpub`	BIP-32 account extended public key	the prospective operator's wallet
`signing_pubkey_hex`	64-hex Ed25519 **public** signing key	the same wallet

The `name` is optional. The trust level is chosen by the enrolling admin, not by the key holder.

Sign-in works like this: the wallet sends a challenge request containing its Ed25519 public key; the node looks the operator up **by that signing key, not by xpub**; the wallet then signs the challenge with the matching private key. So the row must carry the signing public key or the node has nothing to match. The xpub is the row's identity — every endpoint addresses the row by it — and the signing key is the possession proof.

Step 1 — the prospective operator generates a key on their own machine

Shipped path: the in-browser key ceremony — the sanctioned way to generate an SDN key. Dashboard → **Accounts** → **Operator keys** → **"GENERATE A KEY"** dialog (the other half of the paste-an-xpub "APPROVE A KEY" form; both land in the same operator table). Some newer dashboard builds show a paste-only "Enroll a key" form without the generation button; the generation dialog's public output — xpub + signing public key + peer ID + fingerprint — is also reachable from the HD-wallet wallet UIs' copy buttons. The ceremony:

- generates a fresh 24-word recovery phrase entirely in the browser (HD-wallet module, seeded from the browser's own random source);
- derives and shows **only public material**: the account xpub, the peer ID, an xpub fingerprint, and the Ed25519 **public** key;
- never lets the phrase leave the page — not posted, not saved, not written to storage; it disappears when the dialog closes.

The printed signing public key is computed by the exact same unlock path the wallet uses at sign-in, so it is byte-identical to what the phrase presents on login.

CLI alternative (xpub half only). If the operator already has a BIP-39 recovery phrase (e.g. from any offline BIP-39 tool), the CLI derives the standard account xpub at `m/44'/0'/0'`:

```
$ spacedatanetwork derive-xpub
Enter your BIP-39 mnemonic phrase:
--- SDN Identity ---
XPub (BIP-32):
xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi8Aw6ZRc1brxMyWMzG3DSSSSo
ekkudhUd9yLb6qx39T9nMdj

Add to config.yaml:
users:
  - xpub:
"xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi8Aw6ZRc1brxMyWMzG3DSSSS
oekkudhUd9yLb6qx39T9nMdj"
    trust_level: "admin"
    name: "Operator"
xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi8Aw6ZRc1brxMyWMzG3DSSSSo
ekkudhUd9yLb6qx39T9nMdj
```

The bare xpub goes to **standard output** (for scripts); the paste-ready config block goes to standard error. If the module file cannot be found, point at it explicitly: `--wasm <path>` or the `HD_WALLET_WASM_PATH` environment variable.

> **Not yet supported:** `derive-xpub` prints only the xpub. The matching Ed25519 signing public key currently comes from the browser ceremony or the HD-wallet wallet UI (a proposed CLI printing both fields: Section 8). A command named `identity new`, mentioned in some dev environment examples, does **not** exist in the current binary; `identity` offers only `directory`, `export`, `gen-key`, `keys`, `set`, `wizard`.

Step 2 — what gets handed over

The prospective operator passes to an existing administrator **exactly**:

1. the **xpub** (starts with `xpub`),
2. the **64-hex Ed25519 signing public key**,
3. their preferred display **name** (optional).

What must never be handed over, in any channel: the recovery phrase, any private key, the extended private key. If a private value ever enters a chat, a paste, a logged terminal, or an agent transcript, the identity is compromised — generate a new one. (The CLI's custody classification: `key export --format mnemonic` is labeled "THE ENTIRE NODE IDENTITY IN PLAINTEXT" and `base64` / `hex` / `kubo` / `libp2p` formats are labeled "SECRET".) Nothing in enrollment ever asks for a private value; an enrollment request that does is a scam or a bug.

Step 3 — what binds on first login (TOFU)

The signing key is bound **on first successful login — trust on first use**:

- Signing key pre-seeded at enrollment (recommended; it is a required field in the dashboard form): login just works; any other key is refused from day one.
- Row created without a signing key (possible via config): the **first** wallet that completes a successful sign-in binds its key permanently; a conflicting key presented later is refused.
- The binding is a pin — whoever holds the phrase that first signed in owns the account permanently. An admin can pin the key in advance (dashboard form or config `signing_pubkey_hex`), mapping enrollment exactly to the key the holder presented.

Failure modes to avoid:

- **Enrolling the wrong signing key** — the node holds a pin the operator cannot produce; the account can never sign in.
- **Omitting the signing key entirely** — the row is a dead record: the dashboard sends no xpub at sign-in, the node resolves the operator by signing key, and a keyless row matches nothing.

3. Path A — enroll from the CLI (and the config path)

`spacedatanetwork admin init` is the CLI enrollment path, and the one a remote node with no desktop app uses. Run it on the server. It prints the node's public keys and the fingerprint of its encryption key, then enrolls an admin from one of three sources:

SOURCE	THE ADMIN LATER SIGNS IN WITH	WHAT THE NODE STORES
`password`	the same username and password in the dashboard	account xpub (`m/44'/0'/0'`) + sign-in key
`mnemonic`	the same BIP-39 recovery phrase	account xpub + sign-in key
`pubkey`	the wallet that owns the pasted 64-hex Ed25519 key	`ed25519:<hex>` + sign-in key

The password and phrase sources derive the key exactly as the dashboard wallet's `password-fast-v1-legacy` and `bip39-mnemonic-v1-legacy` profiles do (secp256k1 BIP-32 scalar at `m/44'/0'/0'/0/0` used as the Ed25519 seed), locked to hd-wallet-wasm's published vectors. Secrets are read without echo and never stored. Passwords

must be at least 16 characters: the legacy password profile has no key stretching, and an admin's sign-in key is public.

With the daemon running, the admin is added through `POST /api/auth/users` (signed in as the node root). With nothing listening on the admin port, it is written to `auth.db` directly. Non-interactive use:

```
spacedatanetwork admin init --name Ops --source pubkey --pubkey <64-hex> --yes
printf '%s\n' "$PASSWORD" | spacedatanetwork admin init --name Ops \
    --source password --username ops --secret-stdin --yes
```

The older paths below still work.

`accounts trust --xpub` is the only CLI command that touches operator rows, and it **updates an existing operator — it cannot create one**. Proven on the wire against a live node:

```
$ spacedatanetwork accounts trust \
    --xpub
xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi8Aw6ZRc1brxMyWMzG3DSSSSo
ekkudhUd9yLb6qx39T9nMdj \
    --level admin
Error: PUT /api/auth/users/xpub6Bosf...: 400 Bad Request:
{"code":"update_failed","message":"user not found"}
```

An xpub with no row gets "user not found" — nothing is created. The create endpoint (`POST /api/auth/users`) is called by the dashboard form and by `admin init` .

The real CLI-compatible creation path is `config.yaml` — also the path that works before any admin exists (Section 5):

1. Have the prospective operator run `derive-xpub` and hand you the xpub (and, from the browser ceremony or wallet UI, the signing public key).
2. Add a `users:` entry — this is exactly what `derive-xpub` prints:

```
users:
  - xpub:
"xpub6BosfCnifzxcFwrSzQiqu2DBVTshkCXacvNsWGYJVVhhawA7d4R5WSWGFNbi8Aw6ZRc1brxMyWMzG3DSSSS
oekkudhUd9yLb6qx39T9nMdj"
    signing_pubkey_hex:
"e31e29b7c8b76bd08012ae0a917eced0067a183e9f1de980a7ef377115578f93"   # 64 hex chars,
required in practice
    trust_level: "admin"
    name: "Operator"
```

(`signing_pubkey_hex` is optional in the config parser and required in practice, for the same dead-record reason as in the UI. Without it, the node logs that the signing key "will be bound on first login (TOFU)".) 3. Restart the daemon (`spacedatanetwork restart` , or stop/start). 4. The operator signs in; the account is live. Two consequences:

- entries from `config.yaml` **cannot be changed or removed through the API or dashboard** — a config row is re-imposed from the file on every read, so changing trust or name means editing the file and restarting;
- a config row **can** carry trust spellings the API refuses: `never` and `ultimate` .

Once a row exists, the CLI manages it:

```
# change an operator's trust level (or rename)
spacedatanetwork accounts trust --xpub <xpub> --level admin
spacedatanetwork accounts trust --xpub <xpub> --name "New Name"

# view the merged view (peers, operators, and identities that are both)
spacedatanetwork accounts list
```

Every admin command authenticates by signing in as the node's own root key against the live daemon, or pass a session token explicitly with `--session-token <token>` or the `SDN_SESSION_TOKEN` environment variable.

> **Not yet supported:** no CLI command creates an operator row — `accounts add` creates **peer** rows only and explicitly refuses xpubs ("it identifies a wallet account, not a libp2p host"). Proposal: Section 8.

4. Path B — enroll from the dashboard UI

1. Sign in as an administrator.
2. Open **Accounts**, then the **Operators** tab.
3. In the **"Enroll a key"** form enter:
   - **Account xpub** — pasted from the prospective operator's key ceremony or from `derive-xpub`;
   - **Name** — optional, their display name;
   - **Trust** — one of: `unknown`, `marginal`, `standard`, `full`, `admin` (the dropdown offers exactly these; see Section 6);
   - **Signing public key (hex)** — the 64-hex Ed25519 public key from the same ceremony, lowercased; the wallet UIs' convenience copy button produces this directly.
4. Click **Enroll**. The node answers `201` for a new row; a duplicate xpub answers `409 user_exists`.

The form posts exactly the two-value payload above to `POST /api/auth/users` with the session cookie. Any admin session can also edit a row's trust/name/signing key (the row's own edit control in the table) and **Withdraw** a row — see Section 7 for what withdrawal refuses.

5. Bootstrapping: where the first administrator comes from

On a brand-new node, exactly three ways an administrator exists:

1. The node's own key — always. The node derives its identity from its seed. Whoever holds that seed signs in as administrator no matter what the user store contains (the "Node Root" row). This is the fallback that means the operator tier can never be permanently destroyed (Section 7).

2. `config.yaml` **seeding.** Before first start, or at any time, add a `users:` entry with `trust_level: "admin"` as in Section 3. The seeded key signs in immediately.

3. First-admin bootstrap — automatic. When no administrator exists anywhere (no config rows, no database rows at admin tier), the node arms one bootstrap path: the first wallet that completes a full sign-in not matching the node's own root is minted as **"Initial Admin"** at admin trust, and its signing key is bound at that moment. Two constraints are enforced deliberately:

- the xpub must be an account-level xpub — a BIP-32 **master** xpub (depth 0) is *refused* at both the challenge and the minting step, because it would store a key that enumerates an entire wallet. Any xpub from `derive-`

`xpub` (account 0) qualifies;

- the path only exists while `HasAdmin()` is false — the moment any administrator row appears (including a config row), the bootstrap stops arming, and unknown keys get unverifiable challenges instead.

Legacy setup mode: on a daemon with no identity key yet, the terminal banner prints a one-time setup token and the root page redirects to a setup page that creates the initial node identity plus a legacy username/password administrator. Old pre-wallet flow; still present on fresh boxes.

6. Trust levels, and what each one grants

The scale is the classic PGP ownertrust ladder, from hard veto to self-identity:

LEVEL	NUMERIC	GRANTS
`never`	-1	Explicit distrust — hard veto, never overridden
`unknown`	0	No assertion; the fail-closed default
`marginal`	1	Weakest positive assertion
`standard`	2	Normal access; the default for newly added peers
`full`	3	Confident trust
`admin`	4	**Operator tier: administrative API access, can manage other accounts**
`ultimate`	5	Reserved for the node's own identity — never granted to anyone else

Admin grants: all `/api/auth/users*` management (the Enroll/editing surface), the merged accounts view, peer trust management, and the rest of the admin-gated API. Every admin gate is a numeric comparison against tier 4.

Who can assign what:

- **Operators (the API):** only `unknown` through `admin`. The API refuses `never` (operator lockout) and `ultimate` (self-identity) — wiring-proof:

```
$ spacedatanetwork accounts trust --xpub <xpub> --level never
Error: PUT /api/auth/users/<xpub>: 400 Bad Request:
{"code":"invalid_trust_level","message":"trust level \"never\" is not
assignable via this API: must be between \"unknown\" and \"admin\"
(ultimate is reserved for the node's own identity; never as an operator
lockout is not yet supported)"}
```

(The CLI help line listing `never|unknown|marginal|standard|full|admin|ultimate` is the *peer* path's full vocabulary; the operator path always rejects `never` and `ultimate`.)

- **Peers (the CLI `accounts trust --peer-id` path):** the full range, including `never` and `ultimate`, with no ceiling.
- **Config rows:** any spelling is accepted from the file (`never` and `ultimate` included) — the ceiling exists only on the API.

7. Withdrawal, and the guards around it

The dashboard blocks three withdrawals client-side, with the reason in place of a confirmation:

- **From config** — the row is owned by the config file; edit the file and restart instead.
- **Your key** — withdrawing your own key from the node that authenticated you would lock the session's owner out.
- **Last admin** — if this row is the only one at admin tier, withdrawing it would leave the node with no administrator at all.

The server itself has only one gate: the caller must hold an admin session. There is no server-side "self" or "last admin" check — an API caller *can* delete the last row; the guard lives in the UI. Deleting the last row is not permanent: the node re-arms the first-admin bootstrap, so the next wallet to sign in is minted "Initial Admin" again. With the root key's unconditional admission, the operator tier can never be permanently destroyed.

Two more withdrawal facts:

- **Config rows cannot be removed or trust-changed via the API at all** (errors are explicit: "config users cannot be removed" / "config-managed users cannot have trust changed through the API").
- **Live sessions survive a demotion or removal.** A session carries the trust level minted when it was created; it is not re-validated against the row on every request. A withdrawn operator keeps what they already hold in an open session until it expires — withdrawal stops *future* sign-ins immediately (the row is gone from the resolution path) but does not retroactively void open sessions.

8. Formerly unsupported

Both gaps this section used to list are closed by `admin init` (Section 3): it derives and prints the sign-in public key through the same derivation the dashboard wallet uses, and it creates the operator row itself, through the daemon or directly in `auth.db`. The rest of the lifecycle is on the CLI too, through the daemon or `auth.db` the same way:

```
spacedatanetwork admin list                          # admins first
spacedatanetwork admin add                           # same sources and flags as init
spacedatanetwork admin set-trust <account> standard  # unknown | marginal | standard |
full | admin
spacedatanetwork admin remove <account>
```

An account is named by display name, sign-in key or account key; an ambiguous name is refused. The last admin can be neither removed nor demoted (the API answers `409 last_admin`), and rows set in `config.yaml` are edited there.

9. Security notes: what never leaves the operator's machine

- **The recovery phrase.** The browser ceremony generates it locally and drops it when the dialog closes. Never type it into a chat, an email, a ticket, or a terminal command; a phrase that has appeared in a transcript or a log is compromised — generate a new one.
- **Private keys.** The node never asks for them. Enrollment transports exactly two public values: the xpub and the Ed25519 public key.
- **The node's own key material.** At-rest keys are encrypted with a machine-derived key and fail closed if the machine changes. `key export --format mnemonic` prints the entire node identity in plaintext — that command belongs on the box, in a trusted session, not in any pipeline you cannot vouch for.

- **Master xpubs.** A depth-0 master xpub is refused by the first-admin bootstrap because it would enumerate an entire wallet; always enroll the account-level xpub (`derive-xpub` output).
- **No enumeration.** A sign-in attempt with an unknown xpub receives a valid-looking challenge that *can never verify* — probing for whether an account exists gets you silence, not information.
- **Session tokens.** Browser sessions ride an HttpOnly cookie. Scripted administration passes a session token explicitly (`--session-token` / `SDN_SESSION_TOKEN`) — treat it as a bearer credential.

Verified against the shipped binary `sdn-server/spacedatanetwork`, its real help output, the live daemon on `127.0.0.1:7173`, and the node source (`sdn-server/internal/auth/`, `sdn-server/internal/peers/`, `sdn-server/cmd/spacedatanetwork/`, and the dashboard sources under `sdn-js/spaceaware-ui/`).